Library Book Issue Management System

Full Source Code & Technical Implementation Documentation

Technologies: Java 21 • Spring Boot 3 • JDBC • MySQL (library_db) • HTML5 • CSS3 • JavaScript

Component	Implementation Details
Backend Architecture	Java 21, Spring Boot 3.3.4, RESTful Web API, MVC Design Pattern
Persistence & Database	Raw JDBC with HikariCP Connection Pool, MySQL (library_db)
Frontend Technology	Single Page Application (SPA), Semantic HTML5, CSS3 Glassmorphism, Vanilla JS (Fetch API)
Build & Packaging	Apache Maven 3.9.16, Maven Wrapper (mvnw), Dockerfile included
Core Features	Book Catalog, Student Directory, Issue Management, Return & Fine Calculation, Live Metrics Dashboard

1. Database Schema & Seed Data (SQL)

File Location: src/main/resources/db/schema.sql

```
■-- ======================================================================
-- Library Book Issue System - Database Schema & Initial Data
-- ======================================================================

CREATE DATABASE IF NOT EXISTS library_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE library_db;

-- 1. Books Table
CREATE TABLE IF NOT EXISTS books (
    id INT AUTO_INCREMENT PRIMARY KEY,
    isbn VARCHAR(20) NOT NULL UNIQUE,
    title VARCHAR(200) NOT NULL,
    author VARCHAR(150) NOT NULL,
    category VARCHAR(100) NOT NULL,
    total_copies INT NOT NULL DEFAULT 1,
    available_copies INT NOT NULL DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Members Table
CREATE TABLE IF NOT EXISTS members (
    id INT AUTO_INCREMENT PRIMARY KEY,
    member_code VARCHAR(30) NOT NULL UNIQUE,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(25),
    membership_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'ACTIVE'
);

-- 3. Book Issues Table
CREATE TABLE IF NOT EXISTS book_issues (
    id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    issue_date DATE NOT NULL,
    due_date DATE NOT NULL,
```

```
    return_date DATE NULL,
    fine_amount DECIMAL(8,2) DEFAULT 0.00,
    status VARCHAR(20) NOT NULL DEFAULT 'ISSUED',
    notes VARCHAR(255),
    FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE CASCADE,
    FOREIGN KEY (member_id) REFERENCES members(id) ON DELETE CASCADE
);

-- ======================================================================
-- Sample Seed Data
-- ======================================================================

-- Insert Sample Books if table is empty
INSERT IGNORE INTO books (id, isbn, title, author, category, total_copies, available_copies) VALUES
(1, '978-0132350884', 'Clean Code', 'Robert C. Martin', 'Software Engineering', 5, 4),
(2, '978-0201633610', 'Design Patterns', 'Erich Gamma et al.', 'Architecture', 3, 2),
(3, '978-0134685991', 'Effective Java (3rd Edition)', 'Joshua Bloch', 'Java Programming', 4, 4),
(4, '978-0262033848', 'Introduction to Algorithms', 'Thomas H. Cormen', 'Computer Science', 2, 1),
(5, '978-0321356680', 'Effective Java (2nd Edition)', 'Joshua Bloch', 'Java Programming', 2, 2),
(6, '978-0596007126', 'Head First Design Patterns', 'Eric Freeman', 'Software Engineering', 4, 4);

-- Insert Sample Members
INSERT IGNORE INTO members (id, member_code, name, email, phone, membership_date, status) VALUES
(1, 'MEM-1001', 'Alex Johnson', 'alex.j@example.com', '9876543210', '2026-01-15', 'ACTIVE'),
(2, 'MEM-1002', 'Priya Sharma', 'priya.s@example.com', '9811223344', '2026-02-10', 'ACTIVE'),
(3, 'MEM-1003', 'Liam Smith', 'liam.smith@example.com', '9700112233', '2026-03-01', 'ACTIVE');

-- Insert Sample Issues (one active, one returned)
INSERT IGNORE INTO book_issues (id, book_id, member_id, issue_date, due_date, return_date, fine_amount, status, notes) VALUES
(1, 1, 1, DATE_SUB(CURDATE(), INTERVAL 10 DAY), DATE_ADD(CURDATE(), INTERVAL 4 DAY), NULL, 0.00, 'ISSUED', 'First borrow'),
(2, 2, 2, DATE_SUB(CURDATE(), INTERVAL 20 DAY), DATE_SUB(CURDATE(), INTERVAL 6 DAY), NULL, 12.00, 'ISSUED', 'Overdue issue'),
(3, 4, 3, DATE_SUB(CURDATE(), INTERVAL 30 DAY), DATE_SUB(CURDATE(), INTERVAL 16 DAY), DATE_SUB(CURDATE(), INTERVAL 15 DAY), 0.00, 'R
```

2. Project Build Configuration (Maven POM)

File Location: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.4</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>

    <groupId>com.library</groupId>
    <artifactId>library-issue-management</artifactId>
    <version>1.0.0</version>
    <name>Library Book Issue System</name>
    <description>Library Book Issue Management System using Spring Boot, JDBC, MySQL, and HTML/CSS/JS</description>

    <properties>
        <java.version>21</java.version>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- Spring Boot Starter Web: embedded Tomcat & REST support -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring Boot Starter JDBC: DataSource and JdbcTemplate -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc</artifactId>
        </dependency>

        <!-- MySQL Connector / J -->
        <dependency>
```

```
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Spring Boot Starter Test -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Spring Boot Maven Plugin for packaging executable jar -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

3. Application Properties Configuration

File Location: src/main/resources/application.properties

```
# Spring Boot Application Properties
spring.application.name=library-issue-management

# Server Port (Uses cloud PORT if provided, defaults to 8080)
server.port=${PORT:8080}

# MySQL Database Configuration
# Works locally with default values, and can be overridden by Cloud Environment Variables
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/library_db?createDatabaseIfNotExist=true&useSSL=false&allo
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:root}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:monika2007}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Disable banner for cleaner startup logs
spring.main.banner-mode=console
```

4. Main Spring Boot Application Entry Point

File Location: src/main/java/com/library/LibraryApplication.java

```
package com.library;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.core.io.ClassPathResource;
import org.springframework.jdbc.datasource.init.ResourceDatabasePopulator;

import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

/**
 * Main Entry Point for the Library Book Issue Management System.
 * Powered by Spring Boot 3 & Java 21.
 */
@SpringBootApplication
public class LibraryApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }

    /**
     * Initializes the MySQL database tables from schema.sql on first launch if they don't exist.
     */
    @Bean
    public ApplicationRunner initDatabase(DataSource dataSource) {
        return args -> {
            System.out.println("======================================================================");
            System.out.println("   ■ Library Book Issue Management System (Spring Boot + JDBC)");
            System.out.println("======================================================================");
            try (Connection conn = dataSource.getConnection()) {
                System.out.println(" [OK] Successfully connected to MySQL database: library_db");

                try (Statement stmt = conn.createStatement();
                     ResultSet rs = stmt.executeQuery("SHOW TABLES LIKE 'books'")) {
                    if (!rs.next()) {
```

```
                        System.out.println(" [INIT] Creating tables and seeding sample data from schema.sql...");
                        ResourceDatabasePopulator populator = new ResourceDatabasePopulator(new ClassPathResource("db/schema.sql"));
                        populator.execute(dataSource);
                        System.out.println(" [OK] Database schema initialized successfully!");
                    } else {
                        System.out.println(" [OK] Database tables verified.");
                    }
                }
            } catch (Exception e) {
                System.err.println(" [NOTICE] MySQL Connection: " + e.getMessage());
                System.err.println(" [HINT] If credentials do not match, please check 'spring.datasource.password' in 'src/main/reso
            }
            System.out.println("================================================================");
            System.out.println("   ■ Web Dashboard: http://localhost:8080");
            System.out.println("   ■ REST API:      http://localhost:8080/api/stats");
            System.out.println("================================================================");
        };
    }
}
```

5. Book Model (POJO)

File Location: src/main/java/com/library/model/Book.java

```
package com.library.model;

import java.sql.Timestamp;

/**
 * Model representing a Book in the library.
 */
public class Book {
    private int id;
    private String isbn;
    private String title;
    private String author;
    private String category;
    private int totalCopies;
    private int availableCopies;
    private Timestamp createdAt;

    public Book() {}

    public Book(int id, String isbn, String title, String author, String category, int totalCopies, int availableCopies) {
        this.id = id;
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.category = category;
        this.totalCopies = totalCopies;
        this.availableCopies = availableCopies;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
```

```
    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public int getTotalCopies() { return totalCopies; }
    public void setTotalCopies(int totalCopies) { this.totalCopies = totalCopies; }

    public int getAvailableCopies() { return availableCopies; }
    public void setAvailableCopies(int availableCopies) { this.availableCopies = availableCopies; }

    public Timestamp getCreatedAt() { return createdAt; }
    public void setCreatedAt(Timestamp createdAt) { this.createdAt = createdAt; }
}
```

6. Member Model (POJO)

File Location: src/main/java/com/library/model/Member.java

```
package com.library.model;

import java.time.LocalDate;

/**
 * Model representing a Library Member (Student, Faculty, etc.).
 */
public class Member {
    private int id;
    private String memberCode;
    private String name;
    private String email;
    private String phone;
    private String membershipDate;
    private String status;

    public Member() {}

    public Member(int id, String memberCode, String name, String email, String phone, String membershipDate, String status) {
        this.id = id;
        this.memberCode = memberCode;
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.membershipDate = membershipDate;
        this.status = status;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getMemberCode() { return memberCode; }
    public void setMemberCode(String memberCode) { this.memberCode = memberCode; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
```

```
    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }

    public String getMembershipDate() { return membershipDate; }
    public void setMembershipDate(String membershipDate) { this.membershipDate = membershipDate; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}
```

7. BookIssue Transaction Model (POJO)

File Location: src/main/java/com/library/model/BookIssue.java

```
package com.library.model;

import java.math.BigDecimal;

/**
 * Model representing a Book Issue / Borrow Transaction.
 * Contains both foreign keys and joined metadata (bookTitle, memberName) for simple UI display.
 */
public class BookIssue {
    private int id;
    private int bookId;
    private int memberId;
    private String issueDate;
    private String dueDate;
    private String returnDate;
    private BigDecimal fineAmount;
    private String status; // 'ISSUED' or 'RETURNED'
    private String notes;

    // Joined fields for UI convenience
    private String bookTitle;
    private String bookIsbn;
    private String memberName;
    private String memberCode;
    private long daysOverdue;

    public BookIssue() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }

    public int getMemberId() { return memberId; }
    public void setMemberId(int memberId) { this.memberId = memberId; }

    public String getIssueDate() { return issueDate; }
    public void setIssueDate(String issueDate) { this.issueDate = issueDate; }
```

```
    public String getDueDate() { return dueDate; }
    public void setDueDate(String dueDate) { this.dueDate = dueDate; }

    public String getReturnDate() { return returnDate; }
    public void setReturnDate(String returnDate) { this.returnDate = returnDate; }

    public BigDecimal getFineAmount() { return fineAmount; }
    public void setFineAmount(BigDecimal fineAmount) { this.fineAmount = fineAmount; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }

    public String getBookTitle() { return bookTitle; }
    public void setBookTitle(String bookTitle) { this.bookTitle = bookTitle; }

    public String getBookIsbn() { return bookIsbn; }
    public void setBookIsbn(String bookIsbn) { this.bookIsbn = bookIsbn; }

    public String getMemberName() { return memberName; }
    public void setMemberName(String memberName) { this.memberName = memberName; }

    public String getMemberCode() { return memberCode; }
    public void setMemberCode(String memberCode) { this.memberCode = memberCode; }

    public long getDaysOverdue() { return daysOverdue; }
    public void setDaysOverdue(long daysOverdue) { this.daysOverdue = daysOverdue; }
}
```

8. Book Data Access Object (BookDao JDBC)

File Location: src/main/java/com/library/dao/BookDao.java

```
package com.library.dao;

import com.library.model.Book;
import org.springframework.stereotype.Repository;

import javax.sql.DataSource;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * Data Access Object (DAO) for Book entity using standard JDBC.
 * Injected with Spring Boot DataSource for connection management.
 */
@Repository
public class BookDao {

    private final DataSource dataSource;

    public BookDao(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    /**
     * Retrieves all books from the database.
     */
    public List<Book> getAllBooks() throws SQLException {
        List<Book> books = new ArrayList<>();
        String sql = "SELECT * FROM books ORDER BY id DESC";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                books.add(mapResultSetToBook(rs));
            }
        }
        return books;
    }
```

```
    /**
     * Searches books by title, author, category, or ISBN.
     */
    public List<Book> searchBooks(String query) throws SQLException {
        List<Book> books = new ArrayList<>();
        String sql = "SELECT * FROM books WHERE title LIKE ? OR author LIKE ? OR category LIKE ? OR isbn LIKE ? ORDER BY id DESC";
        String pattern = "%" + query + "%";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, pattern);
            ps.setString(2, pattern);
            ps.setString(3, pattern);
            ps.setString(4, pattern);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    books.add(mapResultSetToBook(rs));
                }
            }
        }
        return books;
    }

    /**
     * Gets a single book by ID.
     */
    public Book getBookById(int id) throws SQLException {
        String sql = "SELECT * FROM books WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return mapResultSetToBook(rs);
                }
            }
        }
        return null;
```

```
    }

    /**
     * Adds a new book to the library catalog.
     */
    public boolean addBook(Book book) throws SQLException {
        String sql = "INSERT INTO books (isbn, title, author, category, total_copies, available_copies) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, book.getIsbn());
            ps.setString(2, book.getTitle());
            ps.setString(3, book.getAuthor());
            ps.setString(4, book.getCategory());
            ps.setInt(5, book.getTotalCopies());
            ps.setInt(6, book.getTotalCopies()); // Initially available = total

            int affectedRows = ps.executeUpdate();
            if (affectedRows > 0) {
                try (ResultSet generatedKeys = ps.getGeneratedKeys()) {
                    if (generatedKeys.next()) {
                        book.setId(generatedKeys.getInt(1));
                    }
                }
                return true;
            }
        }
        return false;
    }

    /**
     * Updates an existing book.
     */
    public boolean updateBook(Book book) throws SQLException {
        String sql = "UPDATE books SET isbn = ?, title = ?, author = ?, category = ?, total_copies = ?, available_copies = ? WHERE i
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, book.getIsbn());
            ps.setString(2, book.getTitle());
            ps.setString(3, book.getAuthor());
            ps.setString(4, book.getCategory());
```

```
            ps.setInt(5, book.getTotalCopies());
            ps.setInt(6, book.getAvailableCopies());
            ps.setInt(7, book.getId());
            return ps.executeUpdate() > 0;
        }
    }

    /**
     * Deletes a book by ID.
     */
    public boolean deleteBook(int id) throws SQLException {
        String sql = "DELETE FROM books WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate() > 0;
        }
    }

    private Book mapResultSetToBook(ResultSet rs) throws SQLException {
        Book b = new Book();
        b.setId(rs.getInt("id"));
        b.setIsbn(rs.getString("isbn"));
        b.setTitle(rs.getString("title"));
        b.setAuthor(rs.getString("author"));
        b.setCategory(rs.getString("category"));
        b.setTotalCopies(rs.getInt("total_copies"));
        b.setAvailableCopies(rs.getInt("available_copies"));
        b.setCreatedAt(rs.getTimestamp("created_at"));
        return b;
    }
}
```

9. Member Data Access Object (MemberDao JDBC)

File Location: src/main/java/com/library/dao/MemberDao.java

```
package com.library.dao;

import com.library.model.Member;
import org.springframework.stereotype.Repository;

import javax.sql.DataSource;
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * Data Access Object (DAO) for Member entity using JDBC.
 */
@Repository
public class MemberDao {

    private final DataSource dataSource;

    public MemberDao(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<Member> getAllMembers() throws SQLException {
        List<Member> members = new ArrayList<>();
        String sql = "SELECT * FROM members ORDER BY id DESC";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                members.add(mapResultSetToMember(rs));
            }
        }
        return members;
    }

    public List<Member> searchMembers(String query) throws SQLException {
        List<Member> members = new ArrayList<>();
```

```
        String sql = "SELECT * FROM members WHERE name LIKE ? OR email LIKE ? OR member_code LIKE ? ORDER BY id DESC";
        String pattern = "%" + query + "%";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, pattern);
            ps.setString(2, pattern);
            ps.setString(3, pattern);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    members.add(mapResultSetToMember(rs));
                }
            }
        }
        return members;
    }

    public Member getMemberById(int id) throws SQLException {
        String sql = "SELECT * FROM members WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return mapResultSetToMember(rs);
                }
            }
        }
        return null;
    }

    public boolean addMember(Member member) throws SQLException {
        String sql = "INSERT INTO members (member_code, name, email, phone, membership_date, status) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, member.getMemberCode());
            ps.setString(2, member.getName());
            ps.setString(3, member.getEmail());
            ps.setString(4, member.getPhone());
```

```
            ps.setString(5, member.getMembershipDate() != null ? member.getMembershipDate() : LocalDate.now().toString());
            ps.setString(6, member.getStatus() != null ? member.getStatus() : "ACTIVE");

            int affected = ps.executeUpdate();
            if (affected > 0) {
                try (ResultSet rs = ps.getGeneratedKeys()) {
                    if (rs.next()) {
                        member.setId(rs.getInt(1));
                    }
                }
                return true;
            }
        }
        return false;
    }

    public boolean updateMember(Member member) throws SQLException {
        String sql = "UPDATE members SET member_code = ?, name = ?, email = ?, phone = ?, status = ? WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, member.getMemberCode());
            ps.setString(2, member.getName());
            ps.setString(3, member.getEmail());
            ps.setString(4, member.getPhone());
            ps.setString(5, member.getStatus());
            ps.setInt(6, member.getId());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteMember(int id) throws SQLException {
        String sql = "DELETE FROM members WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate() > 0;
        }
    }

    private Member mapResultSetToMember(ResultSet rs) throws SQLException {
```

```
        Member m = new Member();
        m.setId(rs.getInt("id"));
        m.setMemberCode(rs.getString("member_code"));
        m.setName(rs.getString("name"));
        m.setEmail(rs.getString("email"));
        m.setPhone(rs.getString("phone"));
        m.setMembershipDate(rs.getString("membership_date"));
        m.setStatus(rs.getString("status"));
        return m;
    }
}
```

10. Issue & Return Data Access Object (IssueDao JDBC)

File Location: src/main/java/com/library/dao/IssueDao.java

```
package com.library.dao;

import com.library.model.BookIssue;
import org.springframework.stereotype.Repository;

import javax.sql.DataSource;
import java.math.BigDecimal;
import java.sql.*;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Data Access Object (DAO) for Book Issue and Return transactions.
 * Uses atomic JDBC transactions to guarantee inventory consistency.
 */
@Repository
public class IssueDao {

    private final DataSource dataSource;

    public IssueDao(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    /**
     * Retrieves book issue records, optionally filtered by status ('ALL', 'ISSUED', 'RETURNED', 'OVERDUE').
     */
    public List<BookIssue> getAllIssues(String statusFilter) throws SQLException {
        List<BookIssue> issues = new ArrayList<>();
        StringBuilder sql = new StringBuilder(
            "SELECT bi.*, b.title AS book_title, b.isbn AS book_isbn, m.name AS member_name, m.member_code " +
            "FROM book_issues bi " +
            "JOIN books b ON bi.book_id = b.id " +
            "JOIN members m ON bi.member_id = m.id "
        );
```

```
        if ("ISSUED".equalsIgnoreCase(statusFilter)) {
            sql.append("WHERE bi.status = 'ISSUED' ");
        } else if ("RETURNED".equalsIgnoreCase(statusFilter)) {
            sql.append("WHERE bi.status = 'RETURNED' ");
        } else if ("OVERDUE".equalsIgnoreCase(statusFilter)) {
            sql.append("WHERE bi.status = 'ISSUED' AND bi.due_date < CURDATE() ");
        }
        sql.append("ORDER BY bi.id DESC");

        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql.toString());
             ResultSet rs = ps.executeQuery()) {

            LocalDate today = LocalDate.now();
            while (rs.next()) {
                BookIssue issue = new BookIssue();
                issue.setId(rs.getInt("id"));
                issue.setBookId(rs.getInt("book_id"));
                issue.setMemberId(rs.getInt("member_id"));
                issue.setIssueDate(rs.getString("issue_date"));
                issue.setDueDate(rs.getString("due_date"));
                issue.setReturnDate(rs.getString("return_date"));
                issue.setFineAmount(rs.getBigDecimal("fine_amount"));
                issue.setStatus(rs.getString("status"));
                issue.setNotes(rs.getString("notes"));

                // Joined fields
                issue.setBookTitle(rs.getString("book_title"));
                issue.setBookIsbn(rs.getString("book_isbn"));
                issue.setMemberName(rs.getString("member_name"));
                issue.setMemberCode(rs.getString("member_code"));

                // Real-time overdue calculation
                if ("ISSUED".equalsIgnoreCase(issue.getStatus()) && issue.getDueDate() != null) {
                    LocalDate dueDate = LocalDate.parse(issue.getDueDate());
                    if (today.isAfter(dueDate)) {
                        long days = ChronoUnit.DAYS.between(dueDate, today);
                        issue.setDaysOverdue(days);
                    } else {
                        issue.setDaysOverdue(0);
```

```
                }
            } else {
                issue.setDaysOverdue(0);
            }

            issues.add(issue);
        }
    }
    return issues;
}

/**
 * Issues a book to a member.
 * Wrapped in a JDBC transaction to ensure inventory count decrements reliably.
 */
public boolean issueBook(int bookId, int memberId, int daysDue, String notes) throws SQLException {
    if (daysDue <= 0) {
        daysDue = 14;
    }

    String checkStockSql = "SELECT available_copies FROM books WHERE id = ? FOR UPDATE";
    String insertIssueSql = "INSERT INTO book_issues (book_id, member_id, issue_date, due_date, status, notes) " +
                            "VALUES (?, ?, CURDATE(), DATE_ADD(CURDATE(), INTERVAL ? DAY), 'ISSUED', ?)";
    String decrementStockSql = "UPDATE books SET available_copies = available_copies - 1 WHERE id = ?";

    Connection conn = null;
    try {
        conn = dataSource.getConnection();
        conn.setAutoCommit(false); // Start transaction

        // 1. Check book stock
        int availableCopies = 0;
        try (PreparedStatement checkPs = conn.prepareStatement(checkStockSql)) {
            checkPs.setInt(1, bookId);
            try (ResultSet rs = checkPs.executeQuery()) {
                if (!rs.next()) {
                    throw new SQLException("Book with ID " + bookId + " does not exist.");
                }
                availableCopies = rs.getInt("available_copies");
            }
```

```
            }

            if (availableCopies <= 0) {
                conn.rollback();
                throw new IllegalStateException("Book is currently OUT OF STOCK and cannot be issued.");
            }

            // 2. Insert into book_issues
            try (PreparedStatement insertPs = conn.prepareStatement(insertIssueSql)) {
                insertPs.setInt(1, bookId);
                insertPs.setInt(2, memberId);
                insertPs.setInt(3, daysDue);
                insertPs.setString(4, notes != null ? notes : "");
                insertPs.executeUpdate();
            }

            // 3. Decrement available copies
            try (PreparedStatement updatePs = conn.prepareStatement(decrementStockSql)) {
                updatePs.setInt(1, bookId);
                updatePs.executeUpdate();
            }

            conn.commit(); // Commit transaction
            return true;
        } catch (Exception e) {
            if (conn != null) {
                try { conn.rollback(); } catch (SQLException ignored) {}
            }
            if (e instanceof RuntimeException) throw (RuntimeException) e;
            throw new SQLException(e.getMessage(), e);
        } finally {
            if (conn != null) {
                try { conn.setAutoCommit(true); conn.close(); } catch (SQLException ignored) {}
            }
        }
    }

    /**
     * Processes book return, calculates any overdue fine ($2.00 per late day by default),
     * and restocks the book copy atomically.
```

```
     */
    public Map<String, Object> returnBook(int issueId, Double overrideFine) throws SQLException {
        String findIssueSql = "SELECT bi.*, b.id AS b_id FROM book_issues bi JOIN books b ON bi.book_id = b.id WHERE bi.id = ? FOR U
        String updateIssueSql = "UPDATE book_issues SET return_date = CURDATE(), fine_amount = ?, status = 'RETURNED' WHERE id = ?";
        String incrementStockSql = "UPDATE books SET available_copies = available_copies + 1 WHERE id = ?";

        Connection conn = null;
        Map<String, Object> result = new HashMap<>();

        try {
            conn = dataSource.getConnection();
            conn.setAutoCommit(false);

            int bookId = 0;
            String dueDateStr = null;
            String currentStatus = null;

            // 1. Fetch issue details
            try (PreparedStatement findPs = conn.prepareStatement(findIssueSql)) {
                findPs.setInt(1, issueId);
                try (ResultSet rs = findPs.executeQuery()) {
                    if (!rs.next()) {
                        throw new SQLException("Issue record with ID " + issueId + " not found.");
                    }
                    bookId = rs.getInt("b_id");
                    dueDateStr = rs.getString("due_date");
                    currentStatus = rs.getString("status");
                }
            }

            if ("RETURNED".equalsIgnoreCase(currentStatus)) {
                throw new IllegalStateException("This book has already been returned.");
            }

            // 2. Compute fine
            LocalDate dueDate = LocalDate.parse(dueDateStr);
            LocalDate today = LocalDate.now();
            long daysLate = 0;
            BigDecimal fine;
```

```
            if (today.isAfter(dueDate)) {
                daysLate = ChronoUnit.DAYS.between(dueDate, today);
            }

            if (overrideFine != null && overrideFine >= 0) {
                fine = BigDecimal.valueOf(overrideFine);
            } else {
                fine = BigDecimal.valueOf(daysLate * 2.0);
            }

            // 3. Update issue record
            try (PreparedStatement updatePs = conn.prepareStatement(updateIssueSql)) {
                updatePs.setBigDecimal(1, fine);
                updatePs.setInt(2, issueId);
                updatePs.executeUpdate();
            }

            // 4. Increment available copies
            try (PreparedStatement incPs = conn.prepareStatement(incrementStockSql)) {
                incPs.setInt(1, bookId);
                incPs.executeUpdate();
            }

            conn.commit();

            result.put("success", true);
            result.put("daysLate", daysLate);
            result.put("fineAmount", fine);
            result.put("returnDate", today.toString());
            return result;
        } catch (Exception e) {
            if (conn != null) {
                try { conn.rollback(); } catch (SQLException ignored) {}
            }
            if (e instanceof RuntimeException) throw (RuntimeException) e;
            throw new SQLException(e.getMessage(), e);
        } finally {
            if (conn != null) {
                try { conn.setAutoCommit(true); conn.close(); } catch (SQLException ignored) {}
            }
```

```
        }
    }

    /**
     * Fetches dashboard metric counts.
     */
    public Map<String, Object> getDashboardStats() throws SQLException {
        Map<String, Object> stats = new HashMap<>();

        String booksSql = "SELECT COALESCE(SUM(total_copies), 0) AS total_copies, COALESCE(SUM(available_copies), 0) AS available_co
        String membersSql = "SELECT COUNT(*) AS total_members FROM members WHERE status = 'ACTIVE'";
        String issuesSql = "SELECT COUNT(*) AS active_issues, SUM(CASE WHEN due_date < CURDATE() THEN 1 ELSE 0 END) AS overdue_issue

        try (Connection conn = dataSource.getConnection()) {
            try (PreparedStatement ps = conn.prepareStatement(booksSql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    stats.put("totalCopies", rs.getInt("total_copies"));
                    stats.put("availableCopies", rs.getInt("available_copies"));
                    stats.put("uniqueTitles", rs.getInt("unique_titles"));
                }
            }

            try (PreparedStatement ps = conn.prepareStatement(membersSql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    stats.put("totalMembers", rs.getInt("total_members"));
                }
            }

            try (PreparedStatement ps = conn.prepareStatement(issuesSql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    stats.put("activeIssues", rs.getInt("active_issues"));
                    stats.put("overdueIssues", rs.getInt("overdue_issues"));
                }
            }
        }
        return stats;
    }
```

```
}
```

11. Book REST API Controller

File Location: src/main/java/com/library/controller/BookController.java

```
package com.library.controller;

import com.library.dao.BookDao;
import com.library.model.Book;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.sql.SQLException;
import java.util.List;
import java.util.Map;

/**
 * REST Controller for Book operations.
 */
@RestController
@RequestMapping("/api/books")
@CrossOrigin(origins = "*")
public class BookController {

    private final BookDao bookDao;

    public BookController(BookDao bookDao) {
        this.bookDao = bookDao;
    }

    @GetMapping
    public ResponseEntity<List<Book>> getBooks(@RequestParam(value = "search", required = false) String search) throws SQLException
        if (search != null && !search.trim().isEmpty()) {
            return ResponseEntity.ok(bookDao.searchBooks(search.trim()));
        }
        return ResponseEntity.ok(bookDao.getAllBooks());
    }

    @PostMapping
    public ResponseEntity<?> addBook(@RequestBody Book book) {
        if (book.getTitle() == null || book.getTitle().trim().isEmpty() ||
            book.getIsbn() == null || book.getIsbn().trim().isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Title and ISBN are required."));
        }
```

```
        if (book.getTotalCopies() <= 0) {
            book.setTotalCopies(1);
        }
        book.setAvailableCopies(book.getTotalCopies());

        try {
            boolean success = bookDao.addBook(book);
            if (success) {
                return ResponseEntity.status(HttpStatus.CREATED).body(Map.of(
                        "success", true,
                        "message", "Book added successfully!",
                        "book", book
                ));
            } else {
                return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Failed to add book. ISBN may already exist
            }
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        }
    }

    @DeleteMapping
    public ResponseEntity<?> deleteBook(@RequestParam("id") int id) {
        try {
            boolean success = bookDao.deleteBook(id);
            if (success) {
                return ResponseEntity.ok(Map.of("success", true, "message", "Book deleted successfully."));
            }
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(Map.of("error", true, "message", "Book not found with ID: " + id));
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        }
    }
}
```

12. Member REST API Controller

File Location: src/main/java/com/library/controller/MemberController.java

```
package com.library.controller;

import com.library.dao.MemberDao;
import com.library.model.Member;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.sql.SQLException;
import java.util.List;
import java.util.Map;

/**
 * REST Controller for Member operations.
 */
@RestController
@RequestMapping("/api/members")
@CrossOrigin(origins = "*")
public class MemberController {

    private final MemberDao memberDao;

    public MemberController(MemberDao memberDao) {
        this.memberDao = memberDao;
    }

    @GetMapping
    public ResponseEntity<List<Member>> getMembers(@RequestParam(value = "search", required = false) String search) throws SQLExcept
        if (search != null && !search.trim().isEmpty()) {
            return ResponseEntity.ok(memberDao.searchMembers(search.trim()));
        }
        return ResponseEntity.ok(memberDao.getAllMembers());
    }

    @PostMapping
    public ResponseEntity<?> addMember(@RequestBody Member member) {
        if (member.getName() == null || member.getName().trim().isEmpty() ||
            member.getEmail() == null || member.getEmail().trim().isEmpty()) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Name and Email are required."));
        }
```

```
        if (member.getMemberCode() == null || member.getMemberCode().trim().isEmpty()) {
            member.setMemberCode("MEM-" + System.currentTimeMillis() % 100000);
        }

        try {
            boolean success = memberDao.addMember(member);
            if (success) {
                return ResponseEntity.status(HttpStatus.CREATED).body(Map.of(
                        "success", true,
                        "message", "Member registered successfully!",
                        "member", member
                ));
            } else {
                return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Failed to register member. Email or Code m
            }
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        }
    }

    @DeleteMapping
    public ResponseEntity<?> deleteMember(@RequestParam("id") int id) {
        try {
            boolean success = memberDao.deleteMember(id);
            if (success) {
                return ResponseEntity.ok(Map.of("success", true, "message", "Member deleted successfully."));
            }
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(Map.of("error", true, "message", "Member not found with ID: " + id));
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        }
    }
}
```

13. Issue & Return REST API Controller

File Location: src/main/java/com/library/controller/IssueController.java

```
package com.library.controller;

import com.library.dao.IssueDao;
import com.library.model.BookIssue;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.sql.SQLException;
import java.util.List;
import java.util.Map;

/**
 * REST Controller for Book Issue and Return operations.
 */
@RestController
@RequestMapping("/api/issues")
@CrossOrigin(origins = "*")
public class IssueController {

    private final IssueDao issueDao;

    public IssueController(IssueDao issueDao) {
        this.issueDao = issueDao;
    }

    @GetMapping
    public ResponseEntity<List<BookIssue>> getIssues(@RequestParam(value = "status", defaultValue = "ALL") String status) throws SQL
        return ResponseEntity.ok(issueDao.getAllIssues(status));
    }

    @PostMapping("/borrow")
    public ResponseEntity<?> issueBook(@RequestBody Map<String, Object> payload) {
        if (!payload.containsKey("bookId") || !payload.containsKey("memberId")) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "bookId and memberId are required."));
        }

        try {
            int bookId = Integer.parseInt(payload.get("bookId").toString());
            int memberId = Integer.parseInt(payload.get("memberId").toString());
```

```
            int daysDue = payload.containsKey("daysDue") ? Integer.parseInt(payload.get("daysDue").toString()) : 14;
            String notes = payload.containsKey("notes") && payload.get("notes") != null ? payload.get("notes").toString() : "";

            boolean success = issueDao.issueBook(bookId, memberId, daysDue, notes);
            if (success) {
                return ResponseEntity.ok(Map.of("success", true, "message", "Book issued successfully!"));
            }
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Could not issue book."));
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", e.getMessage()));
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Invalid request parameters."));
        }
    }

    @PostMapping("/return")
    public ResponseEntity<?> returnBook(@RequestBody Map<String, Object> payload) {
        if (!payload.containsKey("issueId")) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "issueId is required."));
        }

        try {
            int issueId = Integer.parseInt(payload.get("issueId").toString());
            Double overrideFine = payload.containsKey("fineAmount") && payload.get("fineAmount") != null
                    ? Double.parseDouble(payload.get("fineAmount").toString())
                    : null;

            Map<String, Object> result = issueDao.returnBook(issueId, overrideFine);
            return ResponseEntity.ok(result);
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", e.getMessage()));
        } catch (SQLException e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body(Map.of("error", true, "message", "Database error: " + e.getMessage()));
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(Map.of("error", true, "message", "Invalid request parameters."));
        }
```

```
    }
}
```

14. Dashboard Stats REST Controller

File Location: src/main/java/com/library/controller/StatsController.java

```java
package com.library.controller;

import com.library.dao.IssueDao;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.sql.SQLException;
import java.util.Map;

/**
 * REST Controller for Dashboard Metrics.
 */
@RestController
@RequestMapping("/api/stats")
@CrossOrigin(origins = "*")
public class StatsController {

    private final IssueDao issueDao;

    public StatsController(IssueDao issueDao) {
        this.issueDao = issueDao;
    }

    @GetMapping
    public ResponseEntity<Map<String, Object>> getStats() throws SQLException {
        return ResponseEntity.ok(issueDao.getDashboardStats());
    }
}
```

15. Frontend Web Application (HTML)

File Location: src/main/resources/static/index.html

```
■<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Library Book Issue Management System</title>
    <link rel="stylesheet" href="/css/style.css">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap" rel="stylesheet">
</head>
<body>
    <div class="app-layout">
        <!-- Sidebar Navigation -->
        <aside class="sidebar">
            <div class="brand">
                <div class="brand-icon">■</div>
                <div class="brand-text">
                    <h2>LibSphere</h2>
                    <span>Issue Management</span>
                </div>
            </div>

            <nav class="nav-menu">
                <button class="nav-item active" data-tab="dashboard">
                    <span class="icon">■</span>
                    <span>Dashboard</span>
                </button>
                <button class="nav-item" data-tab="books">
                    <span class="icon">■</span>
                    <span>Book Catalog</span>
                </button>
                <button class="nav-item" data-tab="members">
                    <span class="icon">■</span>
                    <span>Members</span>
                </button>
                <button class="nav-item" data-tab="issues">
                    <span class="icon">■</span>
                    <span>Issue & Return</span>
                </button>
```

```
            </nav>

            <div class="sidebar-footer">
                <div class="status-indicator">
                    <span class="status-dot"></span>
                    <span id="system-status-text">Server Connected</span>
                </div>
                <small class="version-label">Java 21 • JDBC • MySQL</small>
            </div>
        </aside>

        <!-- Main Content Area -->
        <main class="main-content">
            <!-- Top Bar -->
            <header class="topbar">
                <div class="page-title">
                    <h1 id="current-page-title">Dashboard Overview</h1>
                    <p id="current-page-subtitle">Welcome to your central library management system</p>
                </div>
                <div class="quick-actions">
                    <button class="btn btn-secondary" onclick="openModal('addMemberModal')">+ Add Member</button>
                    <button class="btn btn-secondary" onclick="openModal('addBookModal')">+ Add Book</button>
                    <button class="btn btn-primary" onclick="openIssueModal()">■ Issue Book</button>
                </div>
            </header>

            <div class="content-body">
                <!-- TAB 1: DASHBOARD -->
                <section id="tab-dashboard" class="tab-pane active">
                    <div class="stats-grid">
                        <div class="stat-card card-blue">
                            <div class="stat-icon">■</div>
                            <div class="stat-details">
                                <span class="stat-label">Total Inventory</span>
                                <h3 id="stat-total-books" class="stat-value">--</h3>
                                <span class="stat-desc" id="stat-unique-titles">0 titles registered</span>
                            </div>
                        </div>

                        <div class="stat-card card-green">
```

```
                        <div class="stat-icon">■</div>
                        <div class="stat-details">
                            <span class="stat-label">Available Copies</span>
                            <h3 id="stat-available-books" class="stat-value">--</h3>
                            <span class="stat-desc">Ready to be issued</span>
                        </div>
                    </div>

                    <div class="stat-card card-purple">
                        <div class="stat-icon">■</div>
                        <div class="stat-details">
                            <span class="stat-label">Currently Issued</span>
                            <h3 id="stat-active-issues" class="stat-value">--</h3>
                            <span class="stat-desc">Books currently borrowed</span>
                        </div>
                    </div>

                    <div class="stat-card card-amber">
                        <div class="stat-icon">■■</div>
                        <div class="stat-details">
                            <span class="stat-label">Overdue Books</span>
                            <h3 id="stat-overdue-issues" class="stat-value">--</h3>
                            <span class="stat-desc">Pending return penalty</span>
                        </div>
                    </div>
                </div>

                <div class="dashboard-panels">
                    <!-- Recent Issues Panel -->
                    <div class="panel">
                        <div class="panel-header">
                            <h3>Recent Issue Transactions</h3>
                            <button class="btn-link" onclick="switchTab('issues')">View All →</button>
                        </div>
                        <div class="table-responsive">
                            <table class="data-table">
                                <thead>
                                    <tr>
                                        <th>Book Title</th>
                                        <th>Member</th>
```

```
                                <th>Issue Date</th>
                                <th>Due Date</th>
                                <th>Status</th>
                            </tr>
                        </thead>
                        <tbody id="recent-issues-table-body">
                            <tr><td colspan="5" class="text-center text-muted">Loading data...</td></tr>
                        </tbody>
                    </table>
                </div>
            </div>

            <!-- System Status & Quick Guide Panel -->
            <div class="panel info-panel">
                <div class="panel-header">
                    <h3>Getting Started Guide</h3>
                </div>
                <div class="guide-steps">
                    <div class="step-item">
                        <span class="step-num">1</span>
                        <div>
                            <strong>Verify MySQL Connection</strong>
                            <p>Make sure MySQL service is running and credentials match <code>application.properties</co
                        </div>
                    </div>
                    <div class="step-item">
                        <span class="step-num">2</span>
                        <div>
                            <strong>Manage Books & Members</strong>
                            <p>Add new books to the catalog or register student/faculty members.</p>
                        </div>
                    </div>
                    <div class="step-item">
                        <span class="step-num">3</span>
                        <div>
                            <strong>Issue & Collect Books</strong>
                            <p>Click "Issue Book" to lend a copy. Real-time stock automatically decrements!</p>
                        </div>
                    </div>
                </div>
```

```
                </div>
            </div>
        </section>

        <!-- TAB 2: BOOKS CATALOG -->
        <section id="tab-books" class="tab-pane">
            <div class="panel">
                <div class="panel-controls">
                    <div class="search-box">
                        <span class="search-icon">■</span>
                        <input type="text" id="book-search-input" placeholder="Search by title, author, category, or ISBN...
                    </div>
                    <div class="actions">
                        <button class="btn btn-primary" onclick="openModal('addBookModal')">+ Add New Book</button>
                    </div>
                </div>

                <div class="table-responsive">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th>ID</th>
                                <th>ISBN</th>
                                <th>Title</th>
                                <th>Author</th>
                                <th>Category</th>
                                <th>Total</th>
                                <th>Available</th>
                                <th>Status</th>
                                <th>Action</th>
                            </tr>
                        </thead>
                        <tbody id="books-table-body">
                            <tr><td colspan="9" class="text-center">Loading books...</td></tr>
                        </tbody>
                    </table>
                </div>
            </div>
        </section>
```

```
                <!-- TAB 3: MEMBERS -->
                <section id="tab-members" class="tab-pane">
                    <div class="panel">
                        <div class="panel-controls">
                            <div class="search-box">
                                <span class="search-icon">■</span>
                                <input type="text" id="member-search-input" placeholder="Search by member name, email, or code..." c
                            </div>
                            <div class="actions">
                                <button class="btn btn-primary" onclick="openModal('addMemberModal')">+ Register Member</button>
                            </div>
                        </div>

                        <div class="table-responsive">
                            <table class="data-table">
                                <thead>
                                    <tr>
                                        <th>ID</th>
                                        <th>Member Code</th>
                                        <th>Name</th>
                                        <th>Email</th>
                                        <th>Phone</th>
                                        <th>Joined Date</th>
                                        <th>Status</th>
                                        <th>Action</th>
                                    </tr>
                                </thead>
                                <tbody id="members-table-body">
                                    <tr><td colspan="8" class="text-center">Loading members...</td></tr>
                                </tbody>
                            </table>
                        </div>
                    </div>
                </section>

                <!-- TAB 4: ISSUE & RETURN -->
                <section id="tab-issues" class="tab-pane">
                    <div class="panel">
                        <div class="panel-controls">
                            <div class="filter-pills">
```

```
                            <button class="filter-pill active" onclick="filterIssues('ALL', this)">All Records</button>
                            <button class="filter-pill" onclick="filterIssues('ISSUED', this)">Currently Issued</button>
                            <button class="filter-pill" onclick="filterIssues('OVERDUE', this)">Overdue Only</button>
                            <button class="filter-pill" onclick="filterIssues('RETURNED', this)">Returned</button>
                        </div>
                        <button class="btn btn-primary" onclick="openIssueModal()">■ New Issue</button>
                    </div>

                    <div class="table-responsive">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th>Tx ID</th>
                                    <th>Book</th>
                                    <th>Member</th>
                                    <th>Issue Date</th>
                                    <th>Due Date</th>
                                    <th>Return Date</th>
                                    <th>Fine ($)</th>
                                    <th>Status</th>
                                    <th>Action</th>
                                </tr>
                            </thead>
                            <tbody id="issues-table-body">
                                <tr><td colspan="9" class="text-center">Loading issues...</td></tr>
                            </tbody>
                        </table>
                    </div>
                </div>
            </section>
        </div>
    </main>
</div>

<!-- MODAL: ADD BOOK -->
<div id="addBookModal" class="modal-overlay">
    <div class="modal-card">
        <div class="modal-header">
            <h3>Add New Book to Catalog</h3>
            <button class="modal-close" onclick="closeModal('addBookModal')">&times;</button>
```

```
            </div>
            <form id="addBookForm" onsubmit="submitAddBook(event)">
                <div class="form-group">
                    <label>Book Title *</label>
                    <input type="text" id="book-title" required placeholder="e.g. Clean Code">
                </div>
                <div class="form-row">
                    <div class="form-group">
                        <label>Author *</label>
                        <input type="text" id="book-author" required placeholder="e.g. Robert C. Martin">
                    </div>
                    <div class="form-group">
                        <label>ISBN *</label>
                        <input type="text" id="book-isbn" required placeholder="e.g. 978-0132350884">
                    </div>
                </div>
                <div class="form-row">
                    <div class="form-group">
                        <label>Category *</label>
                        <input type="text" id="book-category" required placeholder="e.g. Software Engineering">
                    </div>
                    <div class="form-group">
                        <label>Total Copies *</label>
                        <input type="number" id="book-copies" required min="1" value="3">
                    </div>
                </div>
                <div class="modal-actions">
                    <button type="button" class="btn btn-secondary" onclick="closeModal('addBookModal')">Cancel</button>
                    <button type="submit" class="btn btn-primary">Save Book</button>
                </div>
            </form>
        </div>
    </div>

    <!-- MODAL: ADD MEMBER -->
    <div id="addMemberModal" class="modal-overlay">
        <div class="modal-card">
            <div class="modal-header">
                <h3>Register Library Member</h3>
                <button class="modal-close" onclick="closeModal('addMemberModal')">&times;</button>
```

```
            </div>
            <form id="addMemberForm" onsubmit="submitAddMember(event)">
                <div class="form-group">
                    <label>Full Name *</label>
                    <input type="text" id="member-name" required placeholder="e.g. Alice Smith">
                </div>
                <div class="form-group">
                    <label>Email Address *</label>
                    <input type="email" id="member-email" required placeholder="e.g. alice@example.com">
                </div>
                <div class="form-row">
                    <div class="form-group">
                        <label>Phone Number</label>
                        <input type="tel" id="member-phone" placeholder="e.g. 9876543210">
                    </div>
                    <div class="form-group">
                        <label>Member Code (Optional)</label>
                        <input type="text" id="member-code" placeholder="Leave blank to auto-generate">
                    </div>
                </div>
                <div class="modal-actions">
                    <button type="button" class="btn btn-secondary" onclick="closeModal('addMemberModal')">Cancel</button>
                    <button type="submit" class="btn btn-primary">Register Member</button>
                </div>
            </form>
        </div>
    </div>

    <!-- MODAL: ISSUE BOOK -->
    <div id="issueBookModal" class="modal-overlay">
        <div class="modal-card">
            <div class="modal-header">
                <h3>Issue Book to Member</h3>
                <button class="modal-close" onclick="closeModal('issueBookModal')">&times;</button>
            </div>
            <form id="issueBookForm" onsubmit="submitIssueBook(event)">
                <div class="form-group">
                    <label>Select Book *</label>
                    <select id="issue-book-select" required>
                        <option value="">-- Choose a book --</option>
```

```
                </select>
            </div>
            <div class="form-group">
                <label>Select Member *</label>
                <select id="issue-member-select" required>
                    <option value="">-- Choose a member --</option>
                </select>
            </div>
            <div class="form-row">
                <div class="form-group">
                    <label>Borrow Period (Days)</label>
                    <select id="issue-period-select">
                        <option value="7">7 Days</option>
                        <option value="14" selected>14 Days (Standard)</option>
                        <option value="21">21 Days</option>
                        <option value="30">30 Days</option>
                    </select>
                </div>
                <div class="form-group">
                    <label>Notes (Optional)</label>
                    <input type="text" id="issue-notes" placeholder="e.g. Exam prep, Reference">
                </div>
            </div>
            <div class="modal-actions">
                <button type="button" class="btn btn-secondary" onclick="closeModal('issueBookModal')">Cancel</button>
                <button type="submit" class="btn btn-primary">Confirm Issue</button>
            </div>
        </form>
    </div>
</div>

<!-- MODAL: RETURN BOOK -->
<div id="returnBookModal" class="modal-overlay">
    <div class="modal-card">
        <div class="modal-header">
            <h3>Return Book</h3>
            <button class="modal-close" onclick="closeModal('returnBookModal')">&times;</button>
        </div>
        <div class="return-details-box">
            <p><strong>Book:</strong> <span id="return-book-title">--</span></p>
```

```
            <p><strong>Borrowed By:</strong> <span id="return-member-name">--</span></p>
            <p><strong>Due Date:</strong> <span id="return-due-date">--</span></p>
            <div id="overdue-banner" class="alert-box" style="display:none;">
                ■■ This book is <strong id="return-days-overdue">0</strong> days overdue!
            </div>
            <div class="form-group" style="margin-top: 15px;">
                <label>Fine Amount ($)</label>
                <input type="number" id="return-fine-input" step="0.5" min="0" value="0.00">
                <small class="text-muted">Standard rate: $2.00 per overdue day</small>
            </div>
        </div>
        <div class="modal-actions">
            <input type="hidden" id="return-issue-id">
            <button type="button" class="btn btn-secondary" onclick="closeModal('returnBookModal')">Cancel</button>
            <button type="button" class="btn btn-primary" onclick="confirmReturnBook()">Complete Return</button>
        </div>
    </div>
</div>

<!-- Toast Notifications Container -->
<div id="toast-container" class="toast-container"></div>

<script src="/js/app.js"></script>
</body>
</html>
```

16. Frontend Stylesheet (CSS)

File Location: src/main/resources/static/css/style.css

```
■/* ======================================================================
   Library Book Issue Management System - Modern CSS
   ====================================================================== */

:root {
    --primary: #4f46e5;
    --primary-hover: #4338ca;
    --primary-light: #eef2ff;
    --secondary: #64748b;
    --success: #10b981;
    --success-light: #ecfdf5;
    --warning: #f59e0b;
    --warning-light: #fffbeb;
    --danger: #ef4444;
    --danger-light: #fef2f2;
    --bg-main: #f8fafc;
    --bg-card: #ffffff;
    --text-primary: #0f172a;
    --text-secondary: #64748b;
    --text-muted: #94a3b8;
    --border: #e2e8f0;
    --border-focus: #818cf8;
    --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
    --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
    --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
    --radius-sm: 6px;
    --radius-md: 10px;
    --radius-lg: 16px;
    --font: 'Plus Jakarta Sans', system-ui, -apple-system, sans-serif;
}

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: var(--font);
}

body {
    background-color: var(--bg-main);
```

```
    color: var(--text-primary);
    min-height: 100vh;
    overflow-x: hidden;
}

/* App Layout */
.app-layout {
    display: flex;
    min-height: 100vh;
}

/* Sidebar */
.sidebar {
    width: 260px;
    background: #0f172a;
    color: #f8fafc;
    display: flex;
    flex-direction: column;
    padding: 24px 16px;
    flex-shrink: 0;
    border-right: 1px solid #1e293b;
}

.brand {
    display: flex;
    align-items: center;
    gap: 12px;
    padding-bottom: 24px;
    margin-bottom: 20px;
    border-bottom: 1px solid #1e293b;
}

.brand-icon {
    font-size: 28px;
    background: #1e293b;
    width: 44px;
    height: 44px;
    display: flex;
    align-items: center;
    justify-content: center;
```

```
    border-radius: var(--radius-md);
}

.brand-text h2 {
    font-size: 18px;
    font-weight: 700;
    color: #ffffff;
    line-height: 1.2;
}

.brand-text span {
    font-size: 11px;
    color: #94a3b8;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.nav-menu {
    display: flex;
    flex-direction: column;
    gap: 6px;
    flex: 1;
}

.nav-item {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 12px 16px;
    border-radius: var(--radius-md);
    background: transparent;
    border: none;
    color: #94a3b8;
    font-size: 14px;
    font-weight: 500;
    cursor: pointer;
    text-align: left;
    transition: all 0.2s ease;
}
```

```
.nav-item .icon {
    font-size: 18px;
}

.nav-item:hover {
    background: #1e293b;
    color: #ffffff;
}

.nav-item.active {
    background: var(--primary);
    color: #ffffff;
    font-weight: 600;
}

.sidebar-footer {
    padding-top: 16px;
    border-top: 1px solid #1e293b;
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.status-indicator {
    display: flex;
    align-items: center;
    gap: 8px;
    font-size: 12px;
    color: #cbd5e1;
}

.status-dot {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background-color: var(--success);
    box-shadow: 0 0 8px var(--success);
}

.version-label {
```

```
    font-size: 11px;
    color: #64748b;
}

/* Main Content */
.main-content {
    flex: 1;
    display: flex;
    flex-direction: column;
    overflow-y: auto;
}

.topbar {
    background: var(--bg-card);
    border-bottom: 1px solid var(--border);
    padding: 20px 32px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    position: sticky;
    top: 0;
    z-index: 10;
}

.page-title h1 {
    font-size: 22px;
    font-weight: 700;
    color: var(--text-primary);
}

.page-title p {
    font-size: 13px;
    color: var(--text-secondary);
    margin-top: 2px;
}

.quick-actions {
    display: flex;
    gap: 10px;
}
```

```
.content-body {
    padding: 32px;
    flex: 1;
}

/* Tab Management */
.tab-pane {
    display: none;
    animation: fadeIn 0.25s ease-in-out;
}

.tab-pane.active {
    display: block;
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(6px); }
    to { opacity: 1; transform: translateY(0); }
}

/* Stat Cards Grid */
.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 20px;
    margin-bottom: 28px;
}

.stat-card {
    background: var(--bg-card);
    border-radius: var(--radius-lg);
    padding: 22px;
    display: flex;
    align-items: flex-start;
    gap: 16px;
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    transition: transform 0.2s, box-shadow 0.2s;
}
```

```
.stat-card:hover {
    transform: translateY(-2px);
    box-shadow: var(--shadow-md);
}

.stat-icon {
    font-size: 28px;
    width: 52px;
    height: 52px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: var(--radius-md);
    background: var(--bg-main);
}

.card-blue .stat-icon { background: #eff6ff; }
.card-green .stat-icon { background: #ecfdf5; }
.card-purple .stat-icon { background: #f5f3ff; }
.card-amber .stat-icon { background: #fffbeb; }

.stat-details {
    display: flex;
    flex-direction: column;
}

.stat-label {
    font-size: 13px;
    font-weight: 500;
    color: var(--text-secondary);
}

.stat-value {
    font-size: 26px;
    font-weight: 800;
    color: var(--text-primary);
    margin: 4px 0;
}
```

```
.stat-desc {
    font-size: 12px;
    color: var(--text-muted);
}

/* Panels */
.panel {
    background: var(--bg-card);
    border-radius: var(--radius-lg);
    border: 1px solid var(--border);
    box-shadow: var(--shadow-sm);
    padding: 24px;
    margin-bottom: 24px;
}

.panel-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 18px;
}

.panel-header h3 {
    font-size: 16px;
    font-weight: 700;
}

.panel-controls {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    margin-bottom: 20px;
    flex-wrap: wrap;
}

.dashboard-panels {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 24px;
```

```
}

@media (max-width: 1024px) {
    .dashboard-panels {
        grid-template-columns: 1fr;
    }
}

/* Guide Steps */
.guide-steps {
    display: flex;
    flex-direction: column;
    gap: 16px;
}

.step-item {
    display: flex;
    gap: 14px;
    align-items: flex-start;
}

.step-num {
    background: var(--primary-light);
    color: var(--primary);
    font-weight: 700;
    width: 28px;
    height: 28px;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 50%;
    flex-shrink: 0;
    font-size: 13px;
}

.step-item p {
    font-size: 12px;
    color: var(--text-secondary);
    margin-top: 2px;
}
```

```
.step-item code {
    background: #f1f5f9;
    padding: 2px 6px;
    border-radius: 4px;
    font-size: 11px;
}

/* Tables */
.table-responsive {
    overflow-x: auto;
}

.data-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 13.5px;
    text-align: left;
}

.data-table th {
    background: #f8fafc;
    color: var(--text-secondary);
    font-weight: 600;
    padding: 12px 16px;
    border-bottom: 1px solid var(--border);
    white-space: nowrap;
}

.data-table td {
    padding: 14px 16px;
    border-bottom: 1px solid var(--border);
    color: var(--text-primary);
    vertical-align: middle;
}

.data-table tbody tr:hover {
    background-color: #f1f5f940;
}
```

```
/* Badges */
.badge {
    display: inline-block;
    padding: 4px 10px;
    border-radius: 20px;
    font-size: 11.5px;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.3px;
}

.badge-success { background: var(--success-light); color: var(--success); }
.badge-warning { background: var(--warning-light); color: #d97706; }
.badge-danger { background: var(--danger-light); color: var(--danger); }
.badge-secondary { background: #f1f5f9; color: var(--text-secondary); }

/* Buttons */
.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    padding: 9px 16px;
    border-radius: var(--radius-sm);
    font-size: 13.5px;
    font-weight: 600;
    cursor: pointer;
    border: none;
    transition: all 0.2s;
    text-decoration: none;
}

.btn-primary {
    background: var(--primary);
    color: #ffffff;
}

.btn-primary:hover {
    background: var(--primary-hover);
}
```

```
.btn-secondary {
    background: #ffffff;
    color: var(--text-primary);
    border: 1px solid var(--border);
}

.btn-secondary:hover {
    background: #f8fafc;
}

.btn-danger-outline {
    background: transparent;
    color: var(--danger);
    border: 1px solid #fca5a5;
    padding: 5px 10px;
    font-size: 12px;
}

.btn-danger-outline:hover {
    background: var(--danger-light);
}

.btn-action {
    padding: 5px 12px;
    font-size: 12px;
    border-radius: 4px;
}

.btn-link {
    background: none;
    border: none;
    color: var(--primary);
    font-weight: 600;
    font-size: 13px;
    cursor: pointer;
}

/* Search Box */
.search-box {
```

```
    display: flex;
    align-items: center;
    background: #ffffff;
    border: 1px solid var(--border);
    border-radius: var(--radius-sm);
    padding: 8px 14px;
    gap: 10px;
    width: 320px;
}

.search-box input {
    border: none;
    outline: none;
    font-size: 13.5px;
    width: 100%;
}

/* Filter Pills */
.filter-pills {
    display: flex;
    gap: 8px;
}

.filter-pill {
    background: var(--bg-main);
    border: 1px solid var(--border);
    border-radius: 20px;
    padding: 6px 14px;
    font-size: 12.5px;
    font-weight: 500;
    cursor: pointer;
    color: var(--text-secondary);
    transition: all 0.2s;
}

.filter-pill.active {
    background: var(--primary);
    color: #ffffff;
    border-color: var(--primary);
}
```

```
/* Modal Windows */
.modal-overlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(15, 23, 42, 0.6);
    backdrop-filter: blur(2px);
    display: none;
    align-items: center;
    justify-content: center;
    z-index: 100;
}

.modal-overlay.open {
    display: flex;
    animation: fadeIn 0.2s ease-in-out;
}

.modal-card {
    background: var(--bg-card);
    width: 100%;
    max-width: 500px;
    border-radius: var(--radius-lg);
    padding: 28px;
    box-shadow: var(--shadow-lg);
}

.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
}

.modal-header h3 {
    font-size: 18px;
    font-weight: 700;
```

```
}

.modal-close {
    background: none;
    border: none;
    font-size: 22px;
    cursor: pointer;
    color: var(--text-muted);
}

.modal-close:hover {
    color: var(--text-primary);
}

.form-group {
    display: flex;
    flex-direction: column;
    gap: 6px;
    margin-bottom: 16px;
}

.form-row {
    display: flex;
    gap: 16px;
}

.form-row .form-group {
    flex: 1;
}

.form-group label {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-secondary);
}

.form-group input,
.form-group select {
    padding: 10px 14px;
    border: 1px solid var(--border);
```

```
    border-radius: var(--radius-sm);
    font-size: 13.5px;
    outline: none;
    transition: border-color 0.2s;
    background: #ffffff;
}

.form-group input:focus,
.form-group select:focus {
    border-color: var(--border-focus);
    box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.15);
}

.modal-actions {
    display: flex;
    justify-content: flex-end;
    gap: 12px;
    margin-top: 24px;
}

.return-details-box {
    background: #f8fafc;
    border: 1px solid var(--border);
    border-radius: var(--radius-md);
    padding: 16px;
    display: flex;
    flex-direction: column;
    gap: 8px;
    font-size: 14px;
}

.alert-box {
    background: #fffbeb;
    border: 1px solid #fde68a;
    color: #92400e;
    padding: 10px 14px;
    border-radius: var(--radius-sm);
    font-size: 13px;
    margin-top: 8px;
}
```

```
/* Toast Notifications */
.toast-container {
    position: fixed;
    top: 24px;
    right: 24px;
    display: flex;
    flex-direction: column;
    gap: 10px;
    z-index: 200;
}

.toast {
    background: #ffffff;
    color: var(--text-primary);
    padding: 14px 20px;
    border-radius: var(--radius-md);
    box-shadow: var(--shadow-lg);
    border-left: 5px solid var(--primary);
    font-size: 13.5px;
    font-weight: 500;
    display: flex;
    align-items: center;
    gap: 10px;
    min-width: 280px;
    animation: slideIn 0.3s ease-out;
}

.toast.toast-success { border-left-color: var(--success); }
.toast.toast-error { border-left-color: var(--danger); }

@keyframes slideIn {
    from { transform: translateX(100%); opacity: 0; }
    to { transform: translateX(0); opacity: 1; }
}

.text-center { text-align: center; }
.text-muted { color: var(--text-muted); }
```

17. Frontend Application Logic (JavaScript)

File Location: src/main/resources/static/js/app.js

```
■/**
 * Library Book Issue Management System - Frontend Controller
 * Uses modern vanilla JavaScript with Async/Await and Fetch API.
 */

const API_BASE = '/api';

// Cache for dropdowns
let cachedBooks = [];
let cachedMembers = [];
let currentIssueFilter = 'ALL';

// --- Initialization ---
document.addEventListener('DOMContentLoaded', () => {
    setupTabNavigation();
    loadDashboard();
});

// --- Tab Navigation ---
function setupTabNavigation() {
    const navItems = document.querySelectorAll('.nav-item');
    navItems.forEach(btn => {
        btn.addEventListener('click', () => {
            const tab = btn.getAttribute('data-tab');
            switchTab(tab);
        });
    });
}

function switchTab(tabName) {
    // Update active nav button
    document.querySelectorAll('.nav-item').forEach(btn => {
        btn.classList.toggle('active', btn.getAttribute('data-tab') === tabName);
    });

    // Update active tab pane
    document.querySelectorAll('.tab-pane').forEach(pane => {
        pane.classList.remove('active');
    });
    const targetPane = document.getElementById(`tab-${tabName}`);
```

```
    if (targetPane) targetPane.classList.add('active');

    // Update header titles
    const titles = {
        'dashboard': ['Dashboard Overview', 'Real-time overview of library transactions and inventory'],
        'books': ['Book Catalog', 'Manage, search, and monitor book stock'],
        'members': ['Members Directory', 'Registered students and faculty members'],
        'issues': ['Issue & Return Desk', 'Track active borrowings, overdue books, and returns']
    };
    if (titles[tabName]) {
        document.getElementById('current-page-title').innerText = titles[tabName][0];
        document.getElementById('current-page-subtitle').innerText = titles[tabName][1];
    }

    // Refresh tab content
    if (tabName === 'dashboard') loadDashboard();
    if (tabName === 'books') loadBooks();
    if (tabName === 'members') loadMembers();
    if (tabName === 'issues') loadIssues(currentIssueFilter);
}

// --- Dashboard ---
async function loadDashboard() {
    try {
        const res = await fetch(`${API_BASE}/stats`);
        if (!res.ok) throw new Error('Failed to load stats');
        const stats = await res.json();

        document.getElementById('stat-total-books').innerText = stats.totalCopies ?? 0;
        document.getElementById('stat-unique-titles').innerText = `${stats.uniqueTitles ?? 0} unique titles`;
        document.getElementById('stat-available-books').innerText = stats.availableCopies ?? 0;
        document.getElementById('stat-active-issues').innerText = stats.activeIssues ?? 0;
        document.getElementById('stat-overdue-issues').innerText = stats.overdueIssues ?? 0;

        // Load recent issues for quick dashboard table
        loadRecentIssues();
    } catch (err) {
        console.error('Error fetching dashboard stats:', err);
    }
}
```

```
async function loadRecentIssues() {
    const tbody = document.getElementById('recent-issues-table-body');
    try {
        const res = await fetch(`${API_BASE}/issues?status=ALL`);
        if (!res.ok) throw new Error();
        const issues = await res.json();

        if (issues.length === 0) {
            tbody.innerHTML = '<tr><td colspan="5" class="text-center text-muted">No transactions found yet.</td></tr>';
            return;
        }

        // Show top 5 recent
        const recent = issues.slice(0, 5);
        tbody.innerHTML = recent.map(iss => {
            const isOverdue = iss.status === 'ISSUED' && iss.daysOverdue > 0;
            const badgeClass = isOverdue ? 'badge-danger' : (iss.status === 'RETURNED' ? 'badge-secondary' : 'badge-success');
            const statusLabel = isOverdue ? `OVERDUE (${iss.daysOverdue}d)` : iss.status;

            return `
                <tr>
                    <td><strong>${escapeHtml(iss.bookTitle)}</strong></td>
                    <td>${escapeHtml(iss.memberName)}</td>
                    <td>${iss.issueDate}</td>
                    <td>${iss.dueDate}</td>
                    <td><span class="badge ${badgeClass}">${statusLabel}</span></td>
                </tr>
            `;
        }).join('');
    } catch (e) {
        tbody.innerHTML = '<tr><td colspan="5" class="text-center text-muted">Could not load recent activity.</td></tr>';
    }
}

// --- Books Management ---
let bookSearchDebounce;
function handleBookSearch() {
    clearTimeout(bookSearchDebounce);
    bookSearchDebounce = setTimeout(() => {
```

```
        const q = document.getElementById('book-search-input').value;
        loadBooks(q);
    }, 300);
}

async function loadBooks(search = '') {
    const tbody = document.getElementById('books-table-body');
    tbody.innerHTML = '<tr><td colspan="9" class="text-center">Loading books...</td></tr>';

    try {
        const url = search ? `${API_BASE}/books?search=${encodeURIComponent(search)}` : `${API_BASE}/books`;
        const res = await fetch(url);
        const books = await res.json();
        cachedBooks = books;

        if (books.length === 0) {
            tbody.innerHTML = '<tr><td colspan="9" class="text-center text-muted">No books found. Click "+ Add New Book" to add one!
            return;
        }

        tbody.innerHTML = books.map(b => {
            const inStock = b.availableCopies > 0;
            const stockBadge = inStock
                ? `<span class="badge badge-success">In Stock (${b.availableCopies})</span>`
                : `<span class="badge badge-danger">Out of Stock</span>`;

            return `
                <tr>
                    <td>#${b.id}</td>
                    <td><code>${escapeHtml(b.isbn)}</code></td>
                    <td><strong>${escapeHtml(b.title)}</strong></td>
                    <td>${escapeHtml(b.author)}</td>
                    <td><span class="badge badge-secondary">${escapeHtml(b.category)}</span></td>
                    <td>${b.totalCopies}</td>
                    <td><strong>${b.availableCopies}</strong></td>
                    <td>${stockBadge}</td>
                    <td>
                        <button class="btn btn-danger-outline" onclick="deleteBook(${b.id}, '${escapeQuote(b.title)}')">Delete</butt
                    </td>
                </tr>
```

```
            `;
        }).join('');
    } catch (err) {
        tbody.innerHTML = '<tr><td colspan="9" class="text-center text-muted">Error loading books. Verify backend connection.</td></
    }
}

async function submitAddBook(event) {
    event.preventDefault();
    const payload = {
        title: document.getElementById('book-title').value.trim(),
        author: document.getElementById('book-author').value.trim(),
        isbn: document.getElementById('book-isbn').value.trim(),
        category: document.getElementById('book-category').value.trim(),
        totalCopies: parseInt(document.getElementById('book-copies').value, 10) || 1
    };

    try {
        const res = await fetch(`${API_BASE}/books`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await res.json();

        if (!res.ok) throw new Error(data.message || 'Failed to add book');

        showToast('Book added successfully!', 'success');
        closeModal('addBookModal');
        document.getElementById('addBookForm').reset();
        loadBooks();
        loadDashboard();
    } catch (err) {
        showToast(err.message, 'error');
    }
}

async function deleteBook(id, title) {
    if (!confirm(`Are you sure you want to delete "${title}"?`)) return;
```

```
    try {
        const res = await fetch(`${API_BASE}/books?id=${id}`, { method: 'DELETE' });
        const data = await res.json();
        if (!res.ok) throw new Error(data.message || 'Could not delete book');

        showToast('Book deleted successfully', 'success');
        loadBooks();
        loadDashboard();
    } catch (err) {
        showToast(err.message, 'error');
    }
}

// --- Members Management ---
let memberSearchDebounce;
function handleMemberSearch() {
    clearTimeout(memberSearchDebounce);
    memberSearchDebounce = setTimeout(() => {
        const q = document.getElementById('member-search-input').value;
        loadMembers(q);
    }, 300);
}

async function loadMembers(search = '') {
    const tbody = document.getElementById('members-table-body');
    tbody.innerHTML = '<tr><td colspan="8" class="text-center">Loading members...</td></tr>';

    try {
        const url = search ? `${API_BASE}/members?search=${encodeURIComponent(search)}` : `${API_BASE}/members`;
        const res = await fetch(url);
        const members = await res.json();
        cachedMembers = members;

        if (members.length === 0) {
            tbody.innerHTML = '<tr><td colspan="8" class="text-center text-muted">No members found. Register a member first!</td></t
            return;
        }

        tbody.innerHTML = members.map(m => `
            <tr>
```

```
                <td>#${m.id}</td>
                <td><code>${escapeHtml(m.memberCode)}</code></td>
                <td><strong>${escapeHtml(m.name)}</strong></td>
                <td>${escapeHtml(m.email)}</td>
                <td>${escapeHtml(m.phone || 'N/A')}</td>
                <td>${m.membershipDate}</td>
                <td><span class="badge badge-success">${m.status}</span></td>
                <td>
                    <button class="btn btn-danger-outline" onclick="deleteMember(${m.id}, '${escapeQuote(m.name)}')">Delete</button>
                </td>
            </tr>
        `).join('');
    } catch (err) {
        tbody.innerHTML = '<tr><td colspan="8" class="text-center text-muted">Error loading members.</td></tr>';
    }
}

async function submitAddMember(event) {
    event.preventDefault();
    const payload = {
        name: document.getElementById('member-name').value.trim(),
        email: document.getElementById('member-email').value.trim(),
        phone: document.getElementById('member-phone').value.trim(),
        memberCode: document.getElementById('member-code').value.trim() || undefined
    };

    try {
        const res = await fetch(`${API_BASE}/members`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await res.json();

        if (!res.ok) throw new Error(data.message || 'Failed to register member');

        showToast('Member registered successfully!', 'success');
        closeModal('addMemberModal');
        document.getElementById('addMemberForm').reset();
        loadMembers();
```

```
        loadDashboard();
    } catch (err) {
        showToast(err.message, 'error');
    }
}

async function deleteMember(id, name) {
    if (!confirm(`Are you sure you want to remove member "${name}"?`)) return;

    try {
        const res = await fetch(`${API_BASE}/members?id=${id}`, { method: 'DELETE' });
        const data = await res.json();
        if (!res.ok) throw new Error(data.message || 'Could not delete member');

        showToast('Member deleted successfully', 'success');
        loadMembers();
        loadDashboard();
    } catch (err) {
        showToast(err.message, 'error');
    }
}

// --- Issue & Return System ---
function filterIssues(status, btnElement) {
    currentIssueFilter = status;
    document.querySelectorAll('.filter-pill').forEach(b => b.classList.remove('active'));
    if (btnElement) btnElement.classList.add('active');
    loadIssues(status);
}

async function loadIssues(status = 'ALL') {
    const tbody = document.getElementById('issues-table-body');
    tbody.innerHTML = '<tr><td colspan="9" class="text-center">Loading transactions...</td></tr>';

    try {
        const res = await fetch(`${API_BASE}/issues?status=${status}`);
        const issues = await res.json();

        if (issues.length === 0) {
            tbody.innerHTML = `<tr><td colspan="9" class="text-center text-muted">No ${status.toLowerCase()} records found.</td></tr
```

```
            return;
        }

        tbody.innerHTML = issues.map(iss => {
            const isOverdue = iss.status === 'ISSUED' && iss.daysOverdue > 0;
            let statusBadge;
            if (iss.status === 'RETURNED') {
                statusBadge = '<span class="badge badge-secondary">RETURNED</span>';
            } else if (isOverdue) {
                statusBadge = `<span class="badge badge-danger">OVERDUE (+${iss.daysOverdue} days)</span>`;
            } else {
                statusBadge = '<span class="badge badge-success">ACTIVE</span>';
            }

            const actionBtn = iss.status === 'ISSUED'
                ? `<button class="btn btn-primary btn-action" onclick="openReturnModal(${iss.id}, '${escapeQuote(iss.bookTitle)}', '
                : `<span class="text-muted">Completed</span>`;

            return `
                <tr>
                    <td>#${iss.id}</td>
                    <td><strong>${escapeHtml(iss.bookTitle)}</strong><br><small class="text-muted">${iss.bookIsbn}</small></td>
                    <td>${escapeHtml(iss.memberName)}<br><small class="text-muted">${iss.memberCode}</small></td>
                    <td>${iss.issueDate}</td>
                    <td>${iss.dueDate}</td>
                    <td>${iss.returnDate ? iss.returnDate : '<span class="text-muted">Pending</span>'}</td>
                    <td><strong>$${Number(iss.fineAmount).toFixed(2)}</strong></td>
                    <td>${statusBadge}</td>
                    <td>${actionBtn}</td>
                </tr>
            `;
        }).join('');
    } catch (err) {
        tbody.innerHTML = '<tr><td colspan="9" class="text-center text-muted">Error loading issue records.</td></tr>';
    }
}

// --- Issue Modal Setup & Submission ---
async function openIssueModal() {
    try {
```

```
        // Fetch fresh books and members
        const [booksRes, membersRes] = await Promise.all([
            fetch(`${API_BASE}/books`),
            fetch(`${API_BASE}/members`)
        ]);
        const books = await booksRes.json();
        const members = await membersRes.json();

        const bookSelect = document.getElementById('issue-book-select');
        const memberSelect = document.getElementById('issue-member-select');

        // Populate Books (only available ones)
        const availableBooks = books.filter(b => b.availableCopies > 0);
        bookSelect.innerHTML = availableBooks.length === 0
            ? '<option value="">No books currently available</option>'
            : '<option value="">-- Choose a book --</option>' + availableBooks.map(b =>
                `<option value="${b.id}">${escapeHtml(b.title)} (${b.availableCopies} available)</option>`
              ).join('');

        // Populate Members
        memberSelect.innerHTML = members.length === 0
            ? '<option value="">No members registered</option>'
            : '<option value="">-- Choose a member --</option>' + members.map(m =>
                `<option value="${m.id}">${escapeHtml(m.name)} (${m.memberCode})</option>`
              ).join('');

        openModal('issueBookModal');
    } catch (err) {
        showToast('Could not load books or members for issuing', 'error');
    }
}

async function submitIssueBook(event) {
    event.preventDefault();
    const payload = {
        bookId: parseInt(document.getElementById('issue-book-select').value, 10),
        memberId: parseInt(document.getElementById('issue-member-select').value, 10),
        daysDue: parseInt(document.getElementById('issue-period-select').value, 10) || 14,
        notes: document.getElementById('issue-notes').value.trim()
    };
```

```
    if (!payload.bookId || !payload.memberId) {
        showToast('Please select both a book and a member', 'error');
        return;
    }

    try {
        const res = await fetch(`${API_BASE}/issues/borrow`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await res.json();

        if (!res.ok) throw new Error(data.message || 'Failed to issue book');

        showToast('Book issued successfully!', 'success');
        closeModal('issueBookModal');
        document.getElementById('issueBookForm').reset();
        loadDashboard();
        loadIssues(currentIssueFilter);
    } catch (err) {
        showToast(err.message, 'error');
    }
}

// --- Return Book Modal & Submission ---
function openReturnModal(issueId, bookTitle, memberName, dueDate, daysOverdue) {
    document.getElementById('return-issue-id').value = issueId;
    document.getElementById('return-book-title').innerText = bookTitle;
    document.getElementById('return-member-name').innerText = memberName;
    document.getElementById('return-due-date').innerText = dueDate;

    const overdueBanner = document.getElementById('overdue-banner');
    const fineInput = document.getElementById('return-fine-input');

    if (daysOverdue > 0) {
        overdueBanner.style.display = 'block';
        document.getElementById('return-days-overdue').innerText = daysOverdue;
        const calculatedFine = (daysOverdue * 2.0).toFixed(2);
```

```
        fineInput.value = calculatedFine;
    } else {
        overdueBanner.style.display = 'none';
        fineInput.value = '0.00';
    }

    openModal('returnBookModal');
}

async function confirmReturnBook() {
    const issueId = parseInt(document.getElementById('return-issue-id').value, 10);
    const fine = parseFloat(document.getElementById('return-fine-input').value) || 0.0;

    try {
        const res = await fetch(`${API_BASE}/issues/return`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ issueId, fineAmount: fine })
        });
        const data = await res.json();

        if (!res.ok) throw new Error(data.message || 'Failed to process return');

        showToast('Book returned successfully!', 'success');
        closeModal('returnBookModal');
        loadDashboard();
        loadIssues(currentIssueFilter);
    } catch (err) {
        showToast(err.message, 'error');
    }
}

// --- Modal Helper Functions ---
function openModal(id) {
    const m = document.getElementById(id);
    if (m) m.classList.add('open');
}

function closeModal(id) {
    const m = document.getElementById(id);
```

```
    if (m) m.classList.remove('open');
}

window.addEventListener('click', (e) => {
    if (e.target.classList.contains('modal-overlay')) {
        e.target.classList.remove('open');
    }
});

// --- Toast Notifications ---
function showToast(message, type = 'success') {
    const container = document.getElementById('toast-container');
    const toast = document.createElement('div');
    toast.className = `toast toast-${type}`;
    const icon = type === 'success' ? '■' : '■';
    toast.innerHTML = `<span>${icon}</span> <span>${escapeHtml(message)}</span>`;
    container.appendChild(toast);

    setTimeout(() => {
        toast.style.opacity = '0';
        toast.style.transform = 'translateX(100%)';
        toast.style.transition = 'all 0.3s ease';
        setTimeout(() => toast.remove(), 300);
    }, 3500);
}

// --- Security String Sanitizers ---
function escapeHtml(str) {
    if (!str) return '';
    return String(str)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
}

function escapeQuote(str) {
    if (!str) return '';
    return String(str).replace(/'/g, "\\'").replace(/"/g, '&quot;');
```

```
}
```